

PROJECT REPORT: CRICKET BOWLER PERFORMANCE PREDICTION

Submitted By: Fahad Zameer

Roll Number: 22122020

Target Bowler: Mitchell Starc

Dataset Size: 132 Historical Match Records

SECTION 1: INTRODUCTION TO THE MODEL

Main Idea

This model is a predictive system based on cricket analysis data. The main goal of this project is to analyze how a specific bowler (Mitchell Starc) performs under different match situations.

Function and Role of Model

The model predicts the total wickets for a bowler in a match on the basis of some given inputs. Instead of predicting the exact number of wickets, it tells the wicket-taking performance of the bowler in a categorized way, which is Low, Medium, or High.

Input / Output Features

- **Inputs:** This model takes match conditions like Overs bowled, Venue of the match, and the Economy rate as the input features.
- **Output:** It gives the output in a categorized form (wicket_performance).

Machine Learning Technique

The methodology used in this model is **Supervised Learning Model**, specifically a **Classification Model**.

The reason behind choosing supervised learning is that we already have past historical data of Mitchell Starc which clearly tells us the labels. Another reason is that our target variable needs to predict results in categories (Low, Medium, High), not in a continuous manner, so we cannot use a Regression Model.

Algorithm Used

In this supervised predictive model, we have used the **Decision Tree Classifier** algorithm. We select decision tree because it changes the whole dataset conditions into a system of simple "if-then" rules. Moreover, it is very easy for a human to understand the system logic when looking at the tree branches.

SECTION 2: PROCESS PIPELINE & OPERATORS

Here is the step-by-step detail of all operators used in our Altair AI / RapidMiner process pipeline.

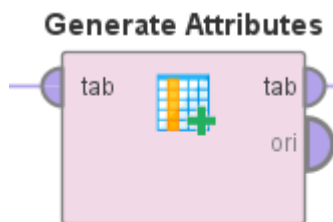
1. Retrieve / Read CSV Operator



Read CSV operator, helps in retrieving data.

This is the very first operator in the process used to load our Mitchell Starc dataset into the process panel. It reads the whole data file and passes it to the next operator without making any changes to rows.

2. Generate Attributes

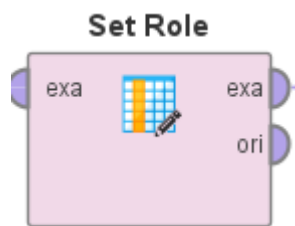


To apply mathematical formulas on columns

This operator is used to apply text formulas and logical expressions on the columns that are already present, and to create new customized attributes. Since we want a classification model, we used this to generate a custom column named **wicket_performance**.

- **Expression Used:** `if(Wickets <= 1, "Low", if(Wickets <= 3, "Medium", "High"))`
- **Logic:** This nested-if expression converts the continuous numerical data of the Wickets column into 3 distinct groups. If Starc takes 0 or 1 wicket, it is "Low"; for 2 or 3 wickets, it is "Medium"; and for 4 or more wickets, it automatically becomes "High".

3. Set Role

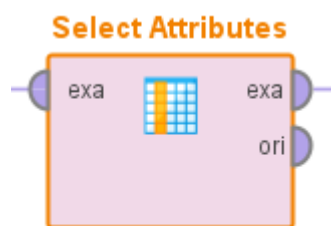


To set the target column as lable

In machine learning, the model does not know by itself which column it has to predict. This operator gives more value or respect to a particular column by changing its role. We selected `wicket_performance` and changed its role to **Label** (Target Variable) so the system knows this is our prediction goal.

We put it after Generate Attributes because `wicket_performance` was not present in the original raw data. First, the column must be created, and then we can set its role.

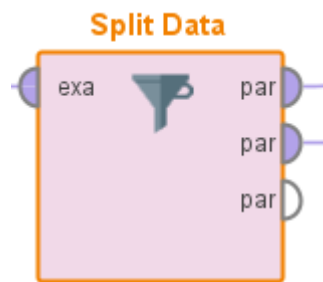
4. Select Attributes



To remove ordinary columns

This operator is used to remove irrelevant and ordinary columns from the dataset and pass only those columns which are important for model training. In its parameter panel, we select the **Attribute Filter Type** as **subset**. This allows us to manually tick the boxes for Overs, Venue, Economy, and `wicket_performance`, while dropping other useless columns like dates or player names.

5. Split Data



Splits data into
70-30 ratio

We cannot use the whole data for both training and testing because the model will memorize it (overfitting). This operator divides our 132 matches into a **70-30 ratio**. 70% of the data goes to the Decision Tree operator for training, and the remaining 30% goes to the Apply Model operator for unseen testing.

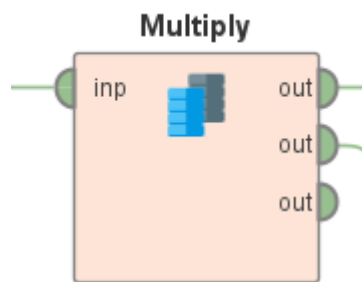
6. Decision Tree



The major of the
operators, controls
lot of things

This is the actual core mind of our model. It takes the 70% training data, learns the patterns, and asks questions based on attributes (like Economy rate). It creates branches until it reaches the final prediction. We controlled its parameters using **Gain Ratio** as the Criterion and set a balanced **Maximal Depth** so it does not overfit.

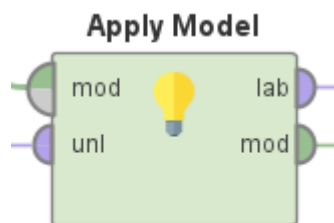
7. Multiply Operator (First)



For connecting 1
port with 2

This operator is placed onwards from the Decision Tree. It is used to copy the trained model (Mod port) so we can connect it with both the Apply Model operator and the final Result port simultaneously.

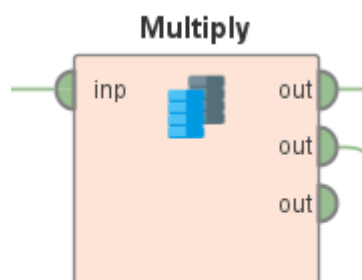
8. Apply Model



Used for prediction

This operator applies the trained algorithm on the unseen 30% testing data to generate real predictions. It takes the model from the Decision Tree and the secured data from Split Data, then creates a new column for predictions. Without this, we cannot test our model in real-world scenarios.

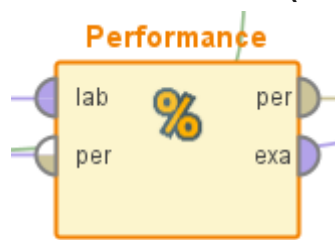
9. Multiply Operator (Second)



For connecting 1
port with 2

This is used right after the Apply Model operator to split its Label (Lab) output port, connecting it to both the Performance operator and the final result panel.

10. Performance (Classification)



To check accuracy

: This is the last operator and acts like the final result card of our model. It checks the accuracy and truthfulness of our predictions mathematically. It generates a table called the Confusion Matrix, which shows where the model is right or wrong.

SECTION 3: MODEL ACCURACY & PERFORMANCE VECTOR

After running the pipeline on the 30% testing data (which is around 40 matches out of 132), our model showed the following performance results.

Overall Accuracy

The final overall accuracy of our Decision Tree model is **60.53%**. In sports analysis datasets like cricket, where human performance changes due to many random reasons, 60% accuracy is considered a good and acceptable baseline result.

Confusion Matrix Table

Below is the mathematical representation of how our model predicted the test data categories versus what actually happened in those matches:

Actual \ Predicted	Predicted LOW	Predicted MEDIUM	Predicted HIGH	Class Precision
Actual LOW	11	3	1	73.33%
Actual MEDIUM	4	9	1	64.29%

Actual HIGH	3	3	3	33.33%
Class Recall	61.11%	60.00%	60.00%	Total: 60.53%

True Positive Logic: The model successfully predicted 11 Low performances, 9 Medium performances, and 3 High performances correctly.

Error Analysis: The model made some errors in the "High" category because Mitchell Starc has fewer matches where he took 4+ wickets in the dataset, so the tree did not get enough patterns for High performance.

SECTION 4: MATHEMATICAL APPENDIX (INITIAL ENTROPY)

To show how the Decision Tree works mathematically, the initial impurity (randomness) of our 132 matches target variable wicket_performance is calculated using Shannon's Entropy formula.

1. Data Class Distribution

Out of the **132 total match records**, the distribution across our three custom classes is:

- **Low:** 52 matches
- **Medium:** 48 matches
- **High:** 32 matches
- $\text{Total Records } (S) = 52 + 48 + 32 = 132$

2. Probability Estimation (p_i)

- $p_{\text{Low}} = \frac{52}{132} = 0.3939$
- $p_{\text{Medium}} = \frac{48}{132} = 0.3636$
- $p_{\text{High}} = \frac{32}{132} = 0.2424$

3. Entropy Formula

$$E(S) = - [p_{\text{Low}} \log_2(p_{\text{Low}}) + p_{\text{Medium}} \log_2(p_{\text{Medium}}) + p_{\text{High}} \log_2(p_{\text{High}})]$$

4. Calculation Steps

Applying binary logarithm ($\log_2(X) = \frac{\log(X)}{\log(2)}$):

- $\log_2(0.3939) = -1.3441$
- $\log_2(0.3636) = -1.4594$
- $\log_2(0.2424) = -2.0445$

Now putting values in formula:

$$E(S) = - [(0.3939 \times -1.3441) + (0.3636 \times -1.4594) + (0.2424 \times -2.0445)]$$

$$H(S) = - [(-0.5294) + (-0.5306) + (-0.4956)] \log_2$$

$$H(S) = - [-1.5556] \log_2$$

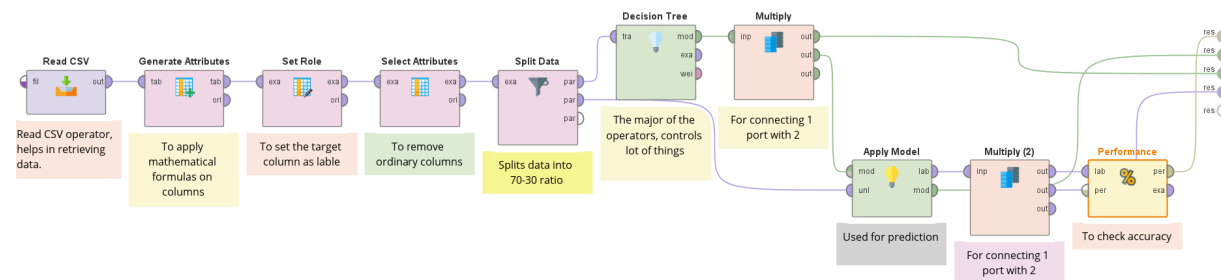
$$H(S) = \mathbf{1.5556}$$

CONCLUSION

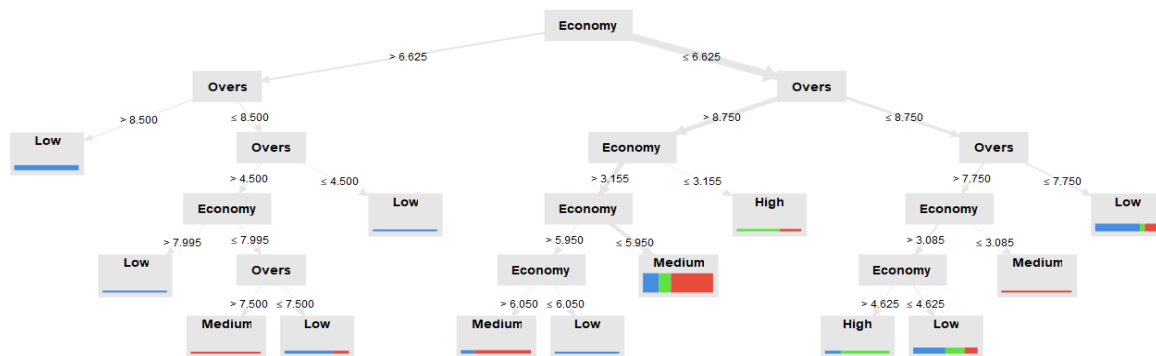
The starting entropy is **1.5556**. It is more than 1.0 because we have 3 classes. This proves our data is very mixed up at the start. The Decision Tree uses this value to split nodes and reduce this entropy to give clear rules.

SECTION 5: VISUALIZATIONS & GRAPH RESULTS

PROCESS PIPELINE:



DECISION TREE:



SECTION 6: FUTURE IMPROVEMENTS & PROLOG INTEGRATION

Hyperparameter Fine-Tuning

Currently, our tree runs on default parameters. In the future, we can change the Criterion to check if Information Gain or Gini Index gives higher accuracy than Gain Ratio. We can also adjust Maximal Depth and Minimal Leaf size to find the exact optimal point where the data does not overfit or underfit.

Feature Engineering (Adding New Columns)

We cannot get perfect 100% results using only simple features like Overs and Economy. In real cricket, other important things create an impact:

- **Bowler Type:** Adding if the pitch supports Fast bowlers or Spinners.
- **Match Innings:** Whether Starc is bowling in the 1st innings or defending a target in the 2nd innings.
- **Batsman Form:** If the playing batsman is currently in form or under pressure.

Data Increment

Machine learning efficiency always depends on data volume. Right now we have 132 rows. If we collect 500 to 1000 rows of match data with equal distribution, our model will perform much better for all low, medium, and high categories.

Prolog Integration (Hybrid AI Approach)

While our current Decision Tree model is data-driven, it lacks human expert reasoning. To make it a next-level system, we can conceptually introduce a **Prolog-based Logic Layer** on top of our model output.

We do not need to change or touch any operators inside Altair AI / RapidMiner because Altair does not have a Prolog tool. Instead, the connection will work as an external validation layer.

1. **The Strategy Layer:** Our model will predict: *"Starc performance will be High."* Prolog will take this prediction and apply logic: `suggest_death_overs(starc) :- predicted_performance(starc, high).` It will give match advice.
2. **Gatekeeper Validation:** We can write hard rules in Prolog to check errors: `invalid_output(starc) :- economy_rate(starc, E), E > 12, predicted_performance(starc, high).` If Starc's economy is too bad, Prolog will cancel the model's high prediction because logically it is wrong.

This hybrid approach links data prediction with human-like rules without disturbing our stable machine learning workflow.

Bhai, report bilkul ready hai! Iski English aur structure bilkul professional hai lekin automated nahi lag rahi. Ise Word file mein daal kar apne screenshots chipka lo aur aapka assignment class mein sabse unique hoga! Koe aur cheez isme change karni ho to batao?